

GOP 통합 데이터베이스 스키마

문서 버전: v1.3

최종 업데이트: 2026-01-06

기준 API 버전: v2.3

데이터베이스: PostgreSQL

목차

1. [개요](#)
2. [Device](#) 관련 테이블
 - 2.1 [devices](#) 테이블 (Base)
 - 2.2 [controllers](#) 테이블
 - 2.3 [sensors](#) 테이블
 - 2.4 [cameras](#) 테이블
3. [DeviceGroup](#) 관련 테이블
 - 3.1 [device_groups](#) 테이블
 - 3.2 [device_group_mappings](#) 테이블
4. [Camera Preset](#) 관련 테이블
 - 4.1 [camera_presets](#) 테이블
 - 4.2 [rois](#) 테이블
 - 4.3 [xy_points](#) 테이블
5. [Event](#) 관련 테이블
 - 5.1 [events](#) 테이블 (Base)
 - 5.2 [detection_events](#) 테이블
 - 5.3 [malfunction_events](#) 테이블
 - 5.4 [connection_events](#) 테이블
 - 5.5 [action_events](#) 테이블
6. [Integration](#) 관련 테이블
 - 6.1 [event_mappings](#) 테이블
 - 6.2 [camera_event_mappings](#) 테이블
 - 6.3 [camera_event_presets](#) 테이블
7. [Server](#) 관련 테이블
 - 7.1 [server_categories](#) 테이블
 - 7.2 [servers](#) 테이블
8. [Enum](#) 타입 정의
9. [ERD](#) 다이어그램
10. [변경 이력](#)

1. 개요

1.1 목적

본 문서는 GOP 통합상황도 시스템의 전체 데이터베이스 스키마를 정의합니다.

1.2 주요 특징

- **데이터베이스**: PostgreSQL
- **명명 규칙**: snake_case (테이블명, 컬럼명)
- **타임스탬프**: 자동 생성/업데이트 (created_at, updated_at)
- **Enum 타입**: PostgreSQL ENUM 사용
- **상속 전략**: Joined Table Inheritance (Device 계층)
- **CASCADE DELETE**: 부모 삭제 시 자식 자동 삭제

1.3 스키마 구성 요약

분류	테이블	설명
Device	devices, controllers, sensors, cameras	장비 관리 (Polymorphic)
DeviceGroup	device_groups, device_group_mappings	장비 그룹 관리 (N:N)
CameraPreset	camera_presets, rois, xy_points	카메라 프리셋 및 ROI
Event	detection_events, malfunction_events, connection_events, action_events	이벤트 관리
Integration	event_mappings, camera_event_mappings, camera_event_presets	이벤트 매핑
Server	server_categories, servers	서버 모니터링

2. Device 관련 테이블

2.1 devices 테이블 (Base)

Device의 공통 필드를 저장하는 기본 테이블입니다. Polymorphic Inheritance의 부모 테이블로 사용됩니다.

PostgreSQL CREATE TABLE

```
-- Enum Types
CREATE TYPE enum_device_category AS ENUM ('controller', 'sensor', 'camera');
CREATE TYPE enum_device_type AS ENUM (
    'NONE', 'Controller', 'Multi', 'Fence', 'Underground', 'Contact', 'PIR',
    'IoController', 'Laser', 'Cable', 'IpCamera', 'SmartSensor', 'SmartSensor2',
    'SmartCompound', 'IpSpeaker', 'Radar', 'OpticalCable', 'Fence_Group'
);
CREATE TYPE enum_device_status AS ENUM ('ACTIVATED', 'ERROR', 'DEACTIVATED');

-- Table
CREATE TABLE devices (
    id SERIAL PRIMARY KEY,
    category_device enum_device_category NOT NULL,
    number_device INTEGER NOT NULL,
    group_device INTEGER NOT NULL,
```

```
name_device VARCHAR(200) NOT NULL,
type_device enum_device_type NOT NULL,
version VARCHAR(50),
status enum_device_status NOT NULL DEFAULT 'ACTIVATED',
created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Indexes
CREATE INDEX idx_devices_id ON devices(id);
CREATE INDEX idx_devices_category_device ON devices(category_device);
CREATE INDEX idx_devices_number_device ON devices(number_device);
CREATE INDEX idx_devices_group_device ON devices(group_device);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
category_device	ENUM	NO	-	Polymorphic Discriminator ('controller', 'sensor', 'camera')
number_device	INTEGER	NO	-	장비 번호
group_device	INTEGER	NO	-	레거시 그룹 ID (호환성 유지)
name_device	VARCHAR(200)	NO	-	장비명
type_device	ENUM	NO	-	장비 유형
version	VARCHAR(50)	YES	NULL	버전 정보
status	ENUM	NO	ACTIVATED	장비 상태
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

2.2 controllers 테이블

Controller 장비의 확장 필드를 저장합니다.

PostgreSQL CREATE TABLE

```
CREATE TABLE controllers (
  id INTEGER PRIMARY KEY REFERENCES devices(id) ON DELETE CASCADE,
  ip_address VARCHAR(50) NOT NULL,
  ip_port INTEGER NOT NULL
);
```

```
-- Indexes
CREATE INDEX idx_controllers_id ON controllers(id);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	INTEGER	NO	-	FK → devices.id (CASCADE DELETE)
ip_address	VARCHAR(50)	NO	-	IP 주소
ip_port	INTEGER	NO	-	포트 번호

2.3 sensors 테이블

Sensor 장비의 확장 필드를 저장합니다.

PostgreSQL CREATE TABLE

```
CREATE TABLE sensors (
  id INTEGER PRIMARY KEY REFERENCES devices(id) ON DELETE CASCADE,
  controller_id INTEGER NOT NULL REFERENCES controllers(id)
);

-- Indexes
CREATE INDEX idx_sensors_id ON sensors(id);
CREATE INDEX idx_sensors_controller_id ON sensors(controller_id);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	INTEGER	NO	-	FK → devices.id (CASCADE DELETE)
controller_id	INTEGER	NO	-	FK → controllers.id

2.4 cameras 테이블

Camera 장비의 확장 필드를 저장합니다.

PostgreSQL CREATE TABLE

```
-- Enum Types
CREATE TYPE enum_camera_mode AS ENUM ('NONE', 'ONVIF', 'EMSTONE_API',
'INNODEP_API', 'ETC');
CREATE TYPE enum_camera_type AS ENUM ('NONE', 'FIXED', 'PTZ');
```

```
-- Table
CREATE TABLE cameras (
  id INTEGER PRIMARY KEY REFERENCES devices(id) ON DELETE CASCADE,
  ip_address VARCHAR(50) NOT NULL,
  ip_port INTEGER NOT NULL,
  user_name VARCHAR(100),
  user_password VARCHAR(200),
  rtsp_uri VARCHAR(500),
  rtsp_port INTEGER NOT NULL,
  mode enum_camera_mode NOT NULL DEFAULT 'NONE',
  category enum_camera_type NOT NULL DEFAULT 'NONE',
  is_record BOOLEAN NOT NULL DEFAULT FALSE,
  hardware_spec JSONB,
  geolocation JSONB
);

-- Indexes
CREATE INDEX idx_cameras_id ON cameras(id);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	INTEGER	NO	-	FK → devices.id (CASCADE DELETE)
ip_address	VARCHAR(50)	NO	-	IP 주소
ip_port	INTEGER	NO	-	포트 번호
user_name	VARCHAR(100)	YES	NULL	사용자명
user_password	VARCHAR(200)	YES	NULL	비밀번호
rtsp_uri	VARCHAR(500)	YES	NULL	RTSP URI
rtsp_port	INTEGER	NO	-	RTSP 포트
mode	ENUM	NO	NONE	카메라 모드
category	ENUM	NO	NONE	카메라 유형 (FIXED/PTZ)
is_record	BOOLEAN	NO	FALSE	녹화 활성화 여부
hardware_spec	JSONB	YES	NULL	하드웨어 사양
geolocation	JSONB	YES	NULL	위치 정보

hardware_spec JSON 구조

```
{
  "name": "GOP 1구역 PTZ 카메라",
  "location": "GOP 1구역 전방 초소",
  "manufacturer": "Hanwha Vision",
  "model": "XNP-6320RH",
```

```

"hardware": "PTZ 32x Optical Zoom",
"firmware": "2.41.01",
"device_id": "HWV-XNP-001",
"mac_address": "00:09:18:AB:CD:EF",
"onvif_version": "2.4.2"
}

```

geolocation JSON 구조

```

{
  "location": "GOP 1구역 전방 초소",
  "latitude": 38.1234,
  "longitude": 127.5678,
  "altitude": 245.5
}

```

3. DeviceGroup 관련 테이블

3.1 device_groups 테이블

장비 그룹을 관리하는 테이블입니다.

PostgreSQL CREATE TABLE

```

CREATE TABLE device_groups (
  id SERIAL PRIMARY KEY,
  name VARCHAR(200) NOT NULL UNIQUE,
  description VARCHAR(500),
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Indexes
CREATE INDEX idx_device_groups_id ON device_groups(id);
CREATE UNIQUE INDEX idx_device_groups_name ON device_groups(name);

```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
name	VARCHAR(200)	NO	-	그룹명 (UNIQUE)
description	VARCHAR(500)	YES	NULL	그룹 설명

필드명	타입	NULL 허용	기본값	설명
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

3.2 device_group_mappings 테이블

Device와 DeviceGroup 간의 N:N 관계를 관리하는 Junction Table입니다.

PostgreSQL CREATE TABLE

```

CREATE TABLE device_group_mappings (
  id SERIAL PRIMARY KEY,
  device_id INTEGER NOT NULL,
  category_device enum_device_category NOT NULL,
  group_id INTEGER NOT NULL REFERENCES device_groups(id) ON DELETE CASCADE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_device_category_group UNIQUE (device_id, category_device,
group_id)
);

-- Indexes
CREATE INDEX idx_device_group_mappings_device_id ON
device_group_mappings(device_id);
CREATE INDEX idx_device_group_mappings_category_device ON
device_group_mappings(category_device);
CREATE INDEX idx_device_group_mappings_group_id ON
device_group_mappings(group_id);

```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
device_id	INTEGER	NO	-	장비 ID
category_device	ENUM	NO	-	장비 카테고리
group_id	INTEGER	NO	-	FK → device_groups.id (CASCADE DELETE)
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간

Constraints

- `UNIQUE(device_id, category_device, group_id)`: 동일 장비-그룹 중복 방지

4. Camera Preset 관련 테이블

4.1 camera_presets 테이블

PTZ 카메라의 프리셋 정보를 저장합니다.

PostgreSQL CREATE TABLE

```
CREATE TABLE camera_presets (  
  id SERIAL PRIMARY KEY,  
  camera_id INTEGER NOT NULL REFERENCES cameras(id) ON DELETE CASCADE,  
  camera_name VARCHAR(200) NOT NULL,  
  preset_index INTEGER NOT NULL,  
  preset_name VARCHAR(100) NOT NULL,  
  touring_time INTEGER NOT NULL DEFAULT 10,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT uq_preset_camera_index UNIQUE (camera_id, preset_index)  
);  
  
-- Indexes  
CREATE INDEX idx_camera_presets_id ON camera_presets(id);  
CREATE INDEX idx_camera_presets_camera_id ON camera_presets(camera_id);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
camera_id	INTEGER	NO	-	FK → cameras.id (CASCADE DELETE)
camera_name	VARCHAR(200)	NO	-	카메라명 (참고용)
preset_index	INTEGER	NO	-	프리셋 인덱스
preset_name	VARCHAR(100)	NO	-	프리셋명
touring_time	INTEGER	NO	10	Home→프리셋 이동 시간 (초)
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

Constraints

- `UNIQUE(camera_id, preset_index)`: 동일 카메라 내 프리셋 인덱스 중복 방지

4.2 rois 테이블

ROI (Region of Interest) 정보를 저장합니다.

PostgreSQL CREATE TABLE

```
CREATE TABLE rois (  
  id SERIAL PRIMARY KEY,  
  preset_id INTEGER NOT NULL REFERENCES camera_presets(id) ON DELETE CASCADE,  
  name VARCHAR(100) NOT NULL,  
  resolution_width FLOAT NOT NULL,  
  resolution_height FLOAT NOT NULL,  
  is_enable BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Indexes  
CREATE INDEX idx_rois_id ON rois(id);  
CREATE INDEX idx_rois_preset_id ON rois(preset_id);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
preset_id	INTEGER	NO	-	FK → camera_presets.id (CASCADE DELETE)
name	VARCHAR(100)	NO	-	ROI 이름
resolution_width	FLOAT	NO	-	기준 해상도 너비
resolution_height	FLOAT	NO	-	기준 해상도 높이
is_enable	BOOLEAN	NO	TRUE	활성화 여부
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

4.3 xy_points 테이블

ROI 다각형의 꼭지점 좌표를 저장합니다.

PostgreSQL CREATE TABLE

```
CREATE TABLE xy_points (  
  id SERIAL PRIMARY KEY,  
  roi_id INTEGER NOT NULL REFERENCES rois(id) ON DELETE CASCADE,
```

```
x FLOAT NOT NULL,
y FLOAT NOT NULL,
"order" INTEGER NOT NULL,
created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT uq_xy_point_roi_order UNIQUE (roi_id, "order")
);

-- Indexes
CREATE INDEX idx_xy_points_id ON xy_points(id);
CREATE INDEX idx_xy_points_roi_id ON xy_points(roi_id);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
roi_id	INTEGER	NO	-	FK → rois.id (CASCADE DELETE)
x	FLOAT	NO	-	X 좌표 (0.0~1.0 정규화 또는 픽셀)
y	FLOAT	NO	-	Y 좌표 (0.0~1.0 정규화 또는 픽셀)
order	INTEGER	NO	-	점 순서 (다각형 그리기 순서)
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

Constraints

- `UNIQUE(roi_id, order)`: 동일 ROI 내 순서 중복 방지

5. Event 관련 테이블

PRD 참조: PRD_Event_ActionEvent_Refactoring.md v2.1

- **Joined Table Inheritance**: events (Base) → detection_events, malfunction_events, connection_events (Child)
- **group_event 필드 제거**: 불필요한 중복 필드 제거
- **device_id FK with SET NULL**: Event 연속성 보장
- **device_description**: Device 정보 스냅샷 (자동 동기화)
- **ActionEvent: from_event_id FK**: events.id 단일 FK 참조

5.1 events 테이블 (Base)

모든 이벤트의 공통 속성을 저장하는 기본 테이블입니다. Polymorphic Inheritance의 부모 테이블로 사용됩니다.

v1.2 변경사항: `group_event` 필드 제거됨 (DeviceGroup은 device_id를 통해 조회)

PostgreSQL CREATE TABLE

```
-- Table
CREATE TABLE events (
  id SERIAL PRIMARY KEY,
  category_event VARCHAR(50) NOT NULL,      -- Polymorphic Discriminator
  (detection/malfunction/connection)
  type_event VARCHAR(50) NOT NULL,          -- 이벤트 타입명
  (Intrusion/Fault/Connection)

  -- Device FK (SET NULL on delete - Event 연속성 보장)
  device_id INTEGER REFERENCES devices(id) ON DELETE SET NULL,
  device_description VARCHAR(500),          -- Device 스냅샷 (자동 동기화)

  sequence INTEGER,                         -- v1.3: NULL 허용 (외부 시스템에서 제공
  되지 않을 수 있음)
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Indexes
CREATE INDEX idx_events_id ON events(id);
CREATE INDEX idx_events_category_event ON events(category_event);
CREATE INDEX idx_events_device_id ON events(device_id);
CREATE INDEX idx_events_created_at ON events(created_at);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
category_event	VARCHAR(50)	NO	-	Polymorphic Discriminator (detection/malfunction/connection)
type_event	VARCHAR(50)	NO	-	이벤트 타입명 (Intrusion/Fault/Connection)
device_id	INTEGER	YES	NULL	FK → devices.id (SET NULL on delete)
device_description	VARCHAR(500)	YES	NULL	Device 정보 스냅샷 (자동 동기화)
sequence	INTEGER	YES	NULL	시퀀스 번호 (v1.3: NULL 허용)
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

Polymorphic Identity

category_event	Child 테이블	설명
detection	detection_events	침입 탐지 이벤트
malfunction	malfunction_events	장비 오동작 이벤트
connection	connection_events	장비 연결 이벤트

5.2 detection_events 테이블

침입 탐지 이벤트의 전용 속성을 저장합니다. (events 테이블 상속)

PostgreSQL CREATE TABLE

```
-- Enum Type (참조: GOP_Restful_Api_연동설계.md)
CREATE TYPE enum_detection_type AS ENUM (
    'NONE', 'CABLE_CUTTING', 'CABLE_CONNECTED', 'PIR_SENSOR',
    'THERMAL_SENSOR', 'VIBRATION_SENSOR', 'CONTACT_SENSOR',
    'DISTANCE_SENSOR', 'AI_DETECT'
);

-- Table (Joined Table Inheritance - Child)
CREATE TABLE detection_events (
    id INTEGER PRIMARY KEY REFERENCES events(id) ON DELETE CASCADE,
    action_reported VARCHAR(10) NOT NULL DEFAULT 'False',
    result enum_detection_type NOT NULL
);

-- Indexes
CREATE INDEX idx_detection_events_id ON detection_events(id);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	INTEGER	NO	-	FK/PK → events.id (CASCADE DELETE)
action_reported	VARCHAR(10)	NO	False	조치 보고 여부
result	ENUM	NO	-	탐지 결과 (enum_detection_type)

5.3 malfunction_events 테이블

장비 오동작 이벤트의 전용 속성을 저장합니다. (events 테이블 상속)

PostgreSQL CREATE TABLE

```
-- Enum Type (참조: GOP_Restful_Api_연동설계.md)
CREATE TYPE enum_fault_type AS ENUM (
```

```

    'FAULT_CONTROLLER', 'FAULT_FENCE', 'FAULT_MULTI',
    'FAULT_CABLE_CUTTING', 'FAULT_ETC'
);

-- Table (Joined Table Inheritance - Child)
CREATE TABLE malfunction_events (
    id INTEGER PRIMARY KEY REFERENCES events(id) ON DELETE CASCADE,
    action_reported VARCHAR(10) NOT NULL DEFAULT 'False',
    reason enum_fault_type NOT NULL,
    first_start INTEGER NOT NULL,
    first_end INTEGER NOT NULL,
    second_start INTEGER NOT NULL,
    second_end INTEGER NOT NULL
);

-- Indexes
CREATE INDEX idx_malfunction_events_id ON malfunction_events(id);

```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	INTEGER	NO	-	FK/PK → events.id (CASCADE DELETE)
action_reported	VARCHAR(10)	NO	False	조치 보고 여부
reason	ENUM	NO	-	오동작 원인 (enum_fault_type)
first_start	INTEGER	NO	-	첫 번째 구간 시작
first_end	INTEGER	NO	-	첫 번째 구간 끝
second_start	INTEGER	NO	-	두 번째 구간 시작
second_end	INTEGER	NO	-	두 번째 구간 끝

5.4 connection_events 테이블

장비 연결 이벤트의 전용 속성을 저장합니다. (events 테이블 상속, 추가 필드 없음)

PostgreSQL CREATE TABLE

```

-- Table (Joined Table Inheritance - Child)
CREATE TABLE connection_events (
    id INTEGER PRIMARY KEY REFERENCES events(id) ON DELETE CASCADE
    -- 추가 전용 필드 없음 (Base 필드만 사용)
);

-- Indexes
CREATE INDEX idx_connection_events_id ON connection_events(id);

```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	INTEGER	NO	-	FK/PK → events.id (CASCADE DELETE)

5.5 action_events 테이블

이벤트에 대한 사용자 조치를 저장합니다. `from_event_id`로 BaseEvent(events 테이블)를 단일 FK 참조합니다.

v1.2 변경사항: `from_event` + `from_type_event` → `from_event_id` (단일 FK)

PostgreSQL CREATE TABLE

```
CREATE TABLE action_events (  
  id SERIAL PRIMARY KEY,  
  
  -- 단일 FK로 BaseEvent 참조 (Polymorphic)  
  from_event_id INTEGER REFERENCES events(id) ON DELETE SET NULL,  
  
  type_event VARCHAR(50) NOT NULL DEFAULT 'Action',  
  content VARCHAR(500) NOT NULL,  
  "user" VARCHAR(100) NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Indexes  
CREATE INDEX idx_action_events_id ON action_events(id);  
CREATE INDEX idx_action_events_from_event_id ON action_events(from_event_id);  
CREATE INDEX idx_action_events_user ON action_events("user");  
CREATE INDEX idx_action_events_created_at ON action_events(created_at);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
from_event_id	INTEGER	YES	NULL	FK → events.id (SET NULL on delete)
type_event	VARCHAR(50)	NO	Action	이벤트 유형
content	VARCHAR(500)	NO	-	조치 내용
user	VARCHAR(100)	NO	-	조치한 사용자
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간

필드명	타입	NULL 허용	기본값	설명
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

Event 타입 조회

```
-- ActionEvent에서 원본 이벤트 타입 조회
SELECT ae.*, e.category_event, e.type_event
FROM action_events ae
LEFT JOIN events e ON ae.from_event_id = e.id;
```

6. Integration 관련 테이블

6.1 event_mappings 테이블

이벤트 매핑 정보를 저장합니다. DeviceGroup을 통해 어떤 장비 그룹에서 발생한 이벤트에 어떤 카메라 프리셋을 실행할지 정의합니다.

v1.2 변경사항: group_event (VARCHAR) → device_group_id (FK) 변경

PostgreSQL CREATE TABLE

```
CREATE TABLE event_mappings (
  id SERIAL PRIMARY KEY,
  name_event VARCHAR(100) NOT NULL,

  -- 변경: group_event (VARCHAR) → device_group_id (FK)
  device_group_id INTEGER REFERENCES device_groups(id) ON DELETE SET NULL,

  category_event VARCHAR(50) NOT NULL,      -- 이벤트 카테고리
  (detection/malfunction/connection)
  description VARCHAR(500),
  status BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Indexes
CREATE INDEX idx_event_mappings_id ON event_mappings(id);
CREATE INDEX idx_event_mappings_device_group_id ON
event_mappings(device_group_id);
CREATE INDEX idx_event_mappings_category_event ON event_mappings(category_event);
CREATE INDEX idx_event_mappings_status ON event_mappings(status);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
name_event	VARCHAR(100)	NO	-	이벤트 매핑 이름
device_group_id	INTEGER	YES	NULL	FK → device_groups.id (SET NULL on delete)
category_event	VARCHAR(50)	NO	-	이벤트 카테고리 (detection/malfunction/connection)
description	VARCHAR(500)	YES	NULL	설명
status	BOOLEAN	NO	TRUE	활성화 여부
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

이벤트-카메라 프리셋 연동 흐름

- DetectionEvent 발생 (device_id = 101)
 - Device(101)의 DeviceGroup 목록 조회 (device_group_mappings N:N)
 - EventMapping에서 device_group_id + category_event로 매핑 조회
 - CameraEventMapping → CameraEventPreset 실행

6.2 camera_event_mappings 테이블

카메라 이벤트 매핑 정보를 저장합니다.

PostgreSQL CREATE TABLE

```
-- Enum Type
CREATE TYPE enum_category_event AS ENUM (
    'NONE', 'FENCE_SENSOR_ONLY', 'FENCE_SENSOR_WITH_MULTI_SENSOR',
    'MULTI_SENSOR_ONLY', 'SENSOR_WITH_CAMERA', 'SENSOR_WITH_AI_CAMERA',
    'AI_CAMERA_ONLY', 'CAMERA_ONLY'
);

-- Table
CREATE TABLE camera_event_mappings (
    id SERIAL PRIMARY KEY,
    name_event VARCHAR(200) NOT NULL,
    group_event VARCHAR(100) NOT NULL,
    category_event enum_category_event NOT NULL,
    description VARCHAR(500),
    status BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
```



```

        updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
    );

-- Indexes
CREATE INDEX idx_camera_event_mappings_id ON camera_event_mappings(id);
CREATE INDEX idx_camera_event_mappings_group_event ON
camera_event_mappings(group_event);
CREATE INDEX idx_camera_event_mappings_category_event ON
camera_event_mappings(category_event);

```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
name_event	VARCHAR(200)	NO	-	이벤트 이름
group_event	VARCHAR(100)	NO	-	이벤트 그룹
category_event	ENUM	NO	-	이벤트 카테고리
description	VARCHAR(500)	YES	NULL	설명
status	BOOLEAN	NO	TRUE	활성화 여부
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

6.3 camera_event_presets 테이블

카메라 이벤트 프리셋 정보를 저장합니다.

PostgreSQL CREATE TABLE

```

CREATE TABLE camera_event_presets (
    id SERIAL PRIMARY KEY,
    mapping_id INTEGER NOT NULL REFERENCES camera_event_mappings(id),
    cam_id INTEGER NOT NULL,
    url_live VARCHAR(500),
    url_record VARCHAR(500),
    category VARCHAR(20) NOT NULL,
    preset_id VARCHAR(50),
    preset_time INTEGER NOT NULL DEFAULT 0,
    home_preset INTEGER NOT NULL DEFAULT 0,
    home_time INTEGER NOT NULL DEFAULT 0
);

-- Indexes
CREATE INDEX idx_camera_event_presets_id ON camera_event_presets(id);

```

```
CREATE INDEX idx_camera_event_presets_mapping_id ON
camera_event_presets(mapping_id);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
mapping_id	INTEGER	NO	-	FK → camera_event_mappings.id
cam_id	INTEGER	NO	-	카메라 ID
url_live	VARCHAR(500)	YES	NULL	실시간 스트림 URL
url_record	VARCHAR(500)	YES	NULL	녹화 스트림 URL
category	VARCHAR(20)	NO	-	카테고리
preset_id	VARCHAR(50)	YES	NULL	프리셋 ID
preset_time	INTEGER	NO	0	프리셋 이동 시간
home_preset	INTEGER	NO	0	홈 프리셋 번호
home_time	INTEGER	NO	0	홈 복귀 시간

7. Server 관련 테이블

7.1 server_categories 테이블

서버 카테고리 (상위 그룹)를 관리하는 테이블입니다.

PostgreSQL CREATE TABLE

```
-- Enum Type
CREATE TYPE enum_server_type AS ENUM (
    'VMS', 'NVR_API', 'STREAMING', 'TRANSCODER', 'MEDIA', 'RECORDING', 'PLAYBACK',
    'STORAGE',
    'AI_ANALYSIS', 'AI_TRAINING', 'AI_INFERENCE', 'ANALYTICS',
    'DB_API', 'SPEAKER_API', 'ENCLOSURE_API', 'PIDS_API',
    'WEB', 'AUTH', 'PROXY', 'BROKER', 'GATEWAY', 'PUSH',
    'LOG', 'BACKUP', 'MONITORING',
    'ETC'
);

-- Table
CREATE TABLE server_categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    type_server enum_server_type NOT NULL UNIQUE,
    description VARCHAR(200),
```

```

    sort_order INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Indexes
CREATE INDEX idx_server_categories_id ON server_categories(id);
CREATE INDEX idx_server_categories_type_server ON server_categories(type_server);
CREATE INDEX idx_server_categories_sort_order ON server_categories(sort_order);

```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
name	VARCHAR(100)	NO	-	카테고리 표시명
type_server	ENUM	NO	-	서버 유형 (UNIQUE)
description	VARCHAR(200)	YES	NULL	설명
sort_order	INTEGER	NO	0	정렬 순서
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

7.2 servers 테이블

서버 인스턴스를 관리하는 테이블입니다.

PostgreSQL CREATE TABLE

```

-- Enum Type
CREATE TYPE enum_server_status AS ENUM ('NORMAL', 'WARNING', 'ERROR');

-- Table
CREATE TABLE servers (
    id SERIAL PRIMARY KEY,
    category_id INTEGER NOT NULL REFERENCES server_categories(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    status enum_server_status NOT NULL DEFAULT 'NORMAL',
    ip_address VARCHAR(45) NOT NULL,
    port INTEGER NOT NULL,
    hostname VARCHAR(100),
    cpu_usage DECIMAL(5,2),
    ram_usage DECIMAL(5,2),
    disk_usage DECIMAL(5,2),
    network_throughput VARCHAR(20),
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,

```

```
        updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP
    );

    -- Indexes
    CREATE INDEX idx_servers_id ON servers(id);
    CREATE INDEX idx_servers_category_id ON servers(category_id);
    CREATE INDEX idx_servers_status ON servers(status);
    CREATE INDEX idx_servers_ip_address ON servers(ip_address);
```

필드 정의

필드명	타입	NULL 허용	기본값	설명
id	SERIAL	NO	AUTO	고유 식별자 (PK)
category_id	INTEGER	NO	-	FK → server_categories.id (CASCADE DELETE)
name	VARCHAR(100)	NO	-	서버 인스턴스명
status	ENUM	NO	NORMAL	서버 상태
ip_address	VARCHAR(45)	NO	-	IP 주소 (IPv4/IPv6)
port	INTEGER	NO	-	포트 번호
hostname	VARCHAR(100)	YES	NULL	호스트명
cpu_usage	DECIMAL(5,2)	YES	NULL	CPU 사용률 (%)
ram_usage	DECIMAL(5,2)	YES	NULL	RAM 사용률 (%)
disk_usage	DECIMAL(5,2)	YES	NULL	DISK 사용률 (%)
network_throughput	VARCHAR(20)	YES	NULL	네트워크 처리량
created_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	생성 시간
updated_at	TIMESTAMP	NO	CURRENT_TIMESTAMP	수정 시간

8. Enum 타입 정의

8.1 enum_device_category

장비 카테고리 (Polymorphic Discriminator)

값	설명
controller	Controller 장비
sensor	Sensor 장비
camera	Camera 장비

8.2 enum_device_type

장비 유형 (18종)

값	설명
NONE	미정의
Controller	컨트롤러
Multi	멀티 센서
Fence	펜스 센서
Underground	지중 센서
Contact	접점 센서
PIR	PIR 센서
IoController	IoController
Laser	레이저 센서
Cable	케이블 센서
IpCamera	IP 카메라
SmartSensor	스마트 센서
SmartSensor2	스마트 센서 2
SmartCompound	스마트 컴파운드
IpSpeaker	IP 스피커
Radar	레이더
OpticalCable	광케이블
Fence_Group	펜스 그룹

8.3 enum_device_status

장비 상태 (3종)

값	설명
ACTIVATED	활성화
ERROR	오류
DEACTIVATED	비활성화

8.4 enum_camera_mode

카메라 모드 (5종)

값	설명
NONE	미정의
ONVIF	ONVIF 프로토콜
EMSTONE_API	Emstone API
INNODEP_API	Innodep API
ETC	기타

8.5 enum_camera_type

카메라 유형 (3종)

값	설명
NONE	미정의
FIXED	고정 카메라
PTZ	PTZ 카메라

8.6 enum_category_event

이벤트 카테고리 (8종)

값	설명
NONE	미정의
FENCE_SENSOR_ONLY	펜스센서 단독
FENCE_SENSOR_WITH_MULTI_SENSOR	펜스센서+멀티센서 AND
MULTI_SENSOR_ONLY	멀티센서 단독
SENSOR_WITH_CAMERA	센서+카메라
SENSOR_WITH_AI_CAMERA	센서+AI 카메라
AI_CAMERA_ONLY	AI 카메라 단독
CAMERA_ONLY	카메라 단독

8.7 enum_detection_type

탐지 결과 유형 (9종) - 참조: GOP_Restful_Api_연동설계.md

값	설명
NONE	미정의
CABLE_CUTTING	케이블 절단
CABLE_CONNECTED	케이블 연결

값	설명
PIR_SENSOR	PIR 센서
THERMAL_SENSOR	열화상 센서
VIBRATION_SENSOR	진동 센서
CONTACT_SENSOR	접점 센서
DISTANCE_SENSOR	거리 센서
AI_DETECT	AI 탐지

8.8 enum_fault_type

오동작 원인 유형 (5종) - 참조: GOP_Restful_Api_연동설계.md

값	설명
FAULT_CONTROLLER	제어기 장애
FAULT_FENCE	펜스 장애
FAULT_MULTI	복합 장애
FAULT_CABLE_CUTTING	케이블 절단
FAULT_ETC	기타 장애

8.9 enum_server_type

서버 유형 (26종) - [GOP_서버모니터링_스키마.md](#) 참조

8.10 enum_server_status

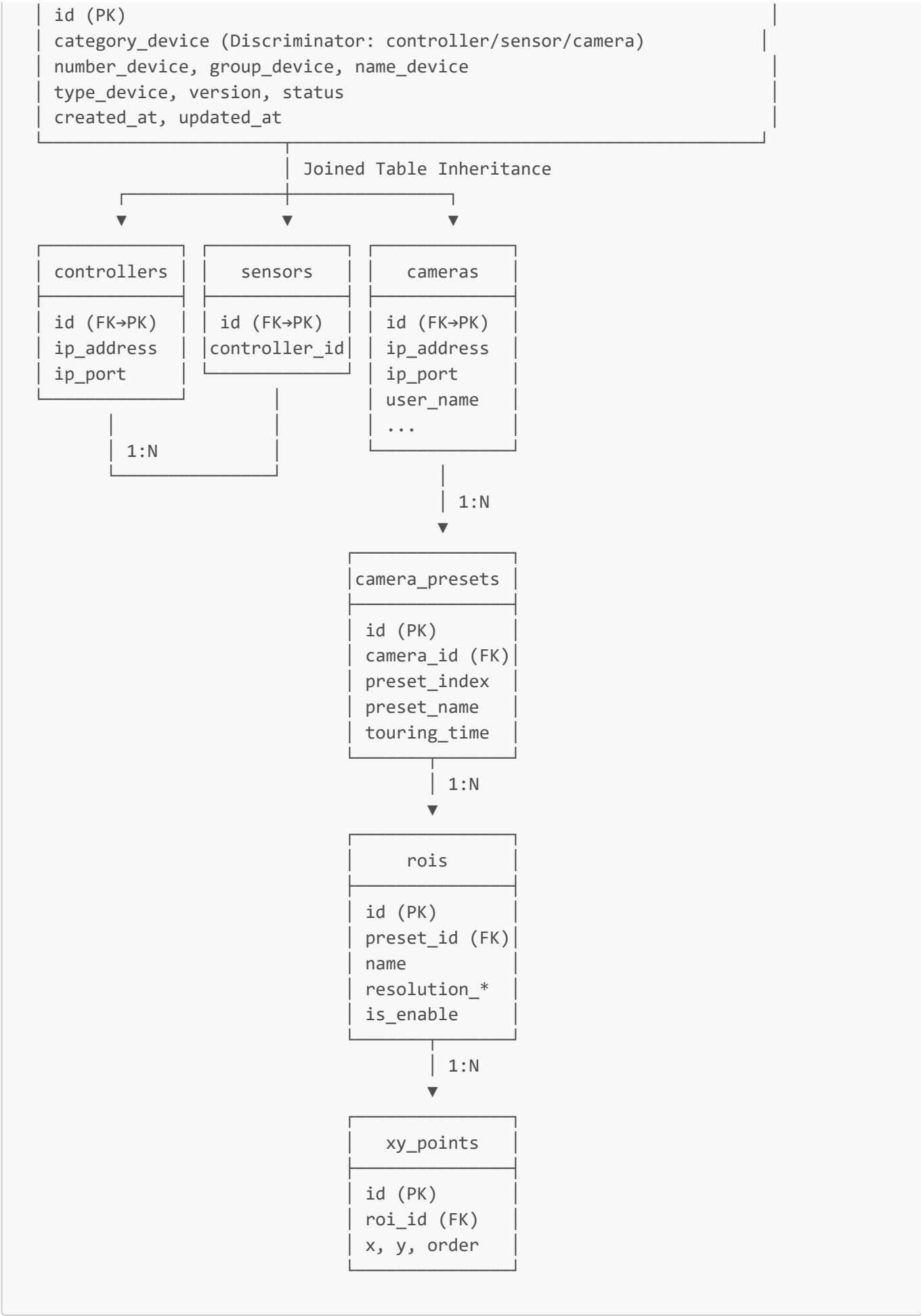
서버 상태 (3종)

값	표시명	색상	설명
NORMAL	정상	녹색 (#10b981)	정상 동작
WARNING	경고	노란색 (#f59e0b)	주의 필요
ERROR	오류	빨간색 (#ef4444)	오류 상태

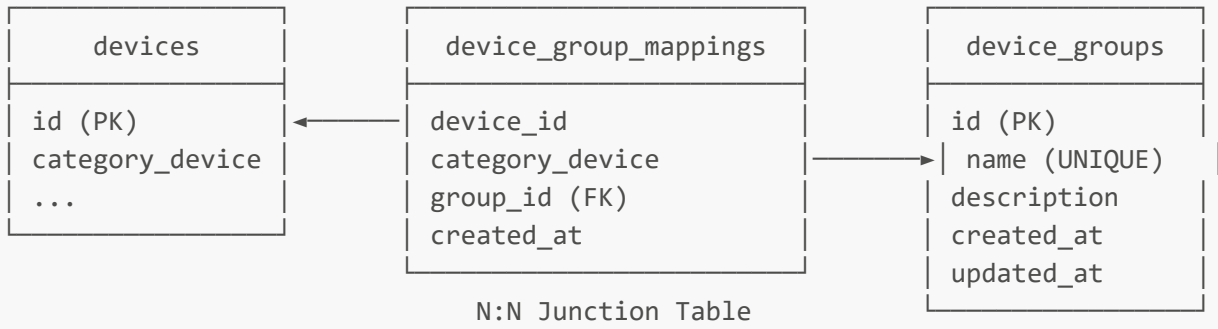
9. ERD 다이어그램

9.1 Device 계층 구조

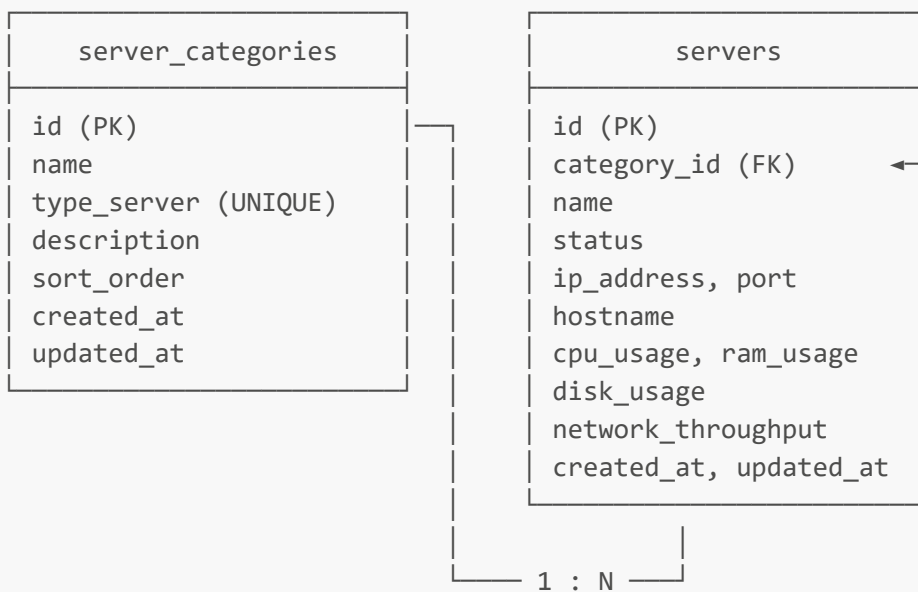




9.2 DeviceGroup N:N 관계

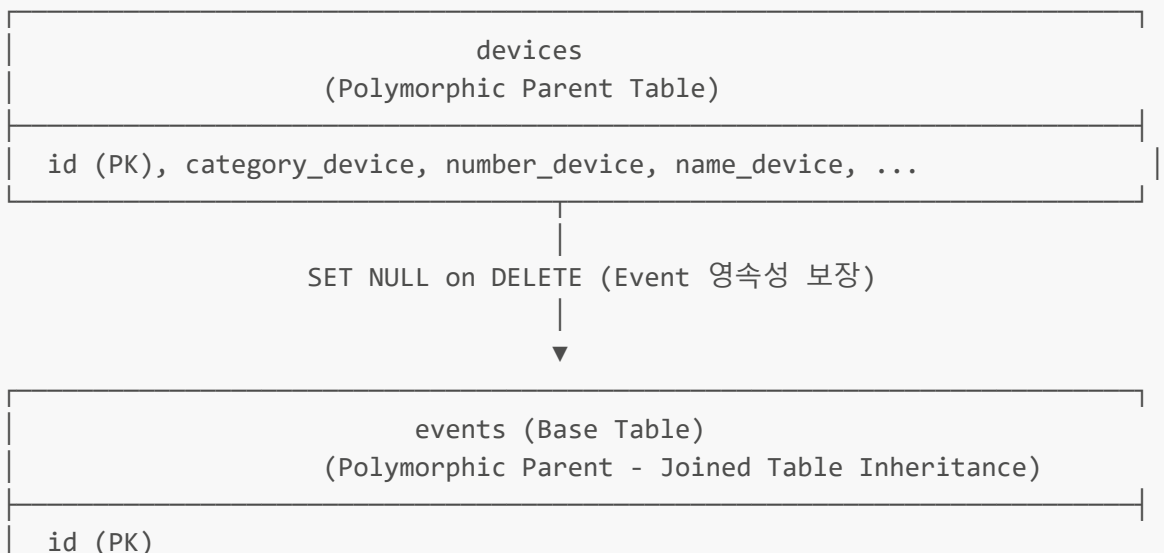


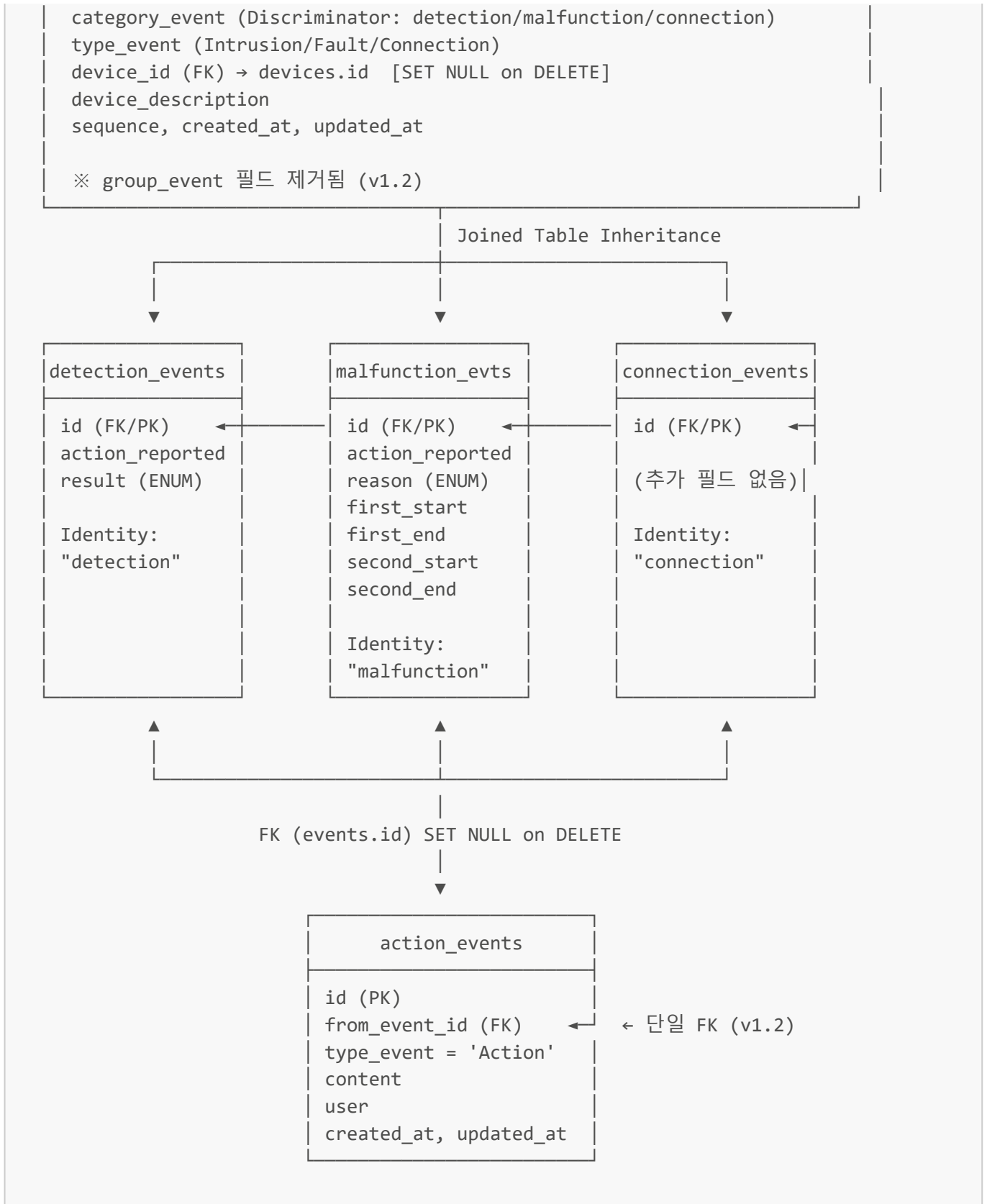
9.3 Server 계층 구조



9.4 Event Joined Table Inheritance 구조

v1.2 변경사항: Joined Table Inheritance 적용, group_event 필드 제거

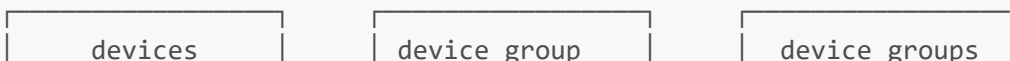


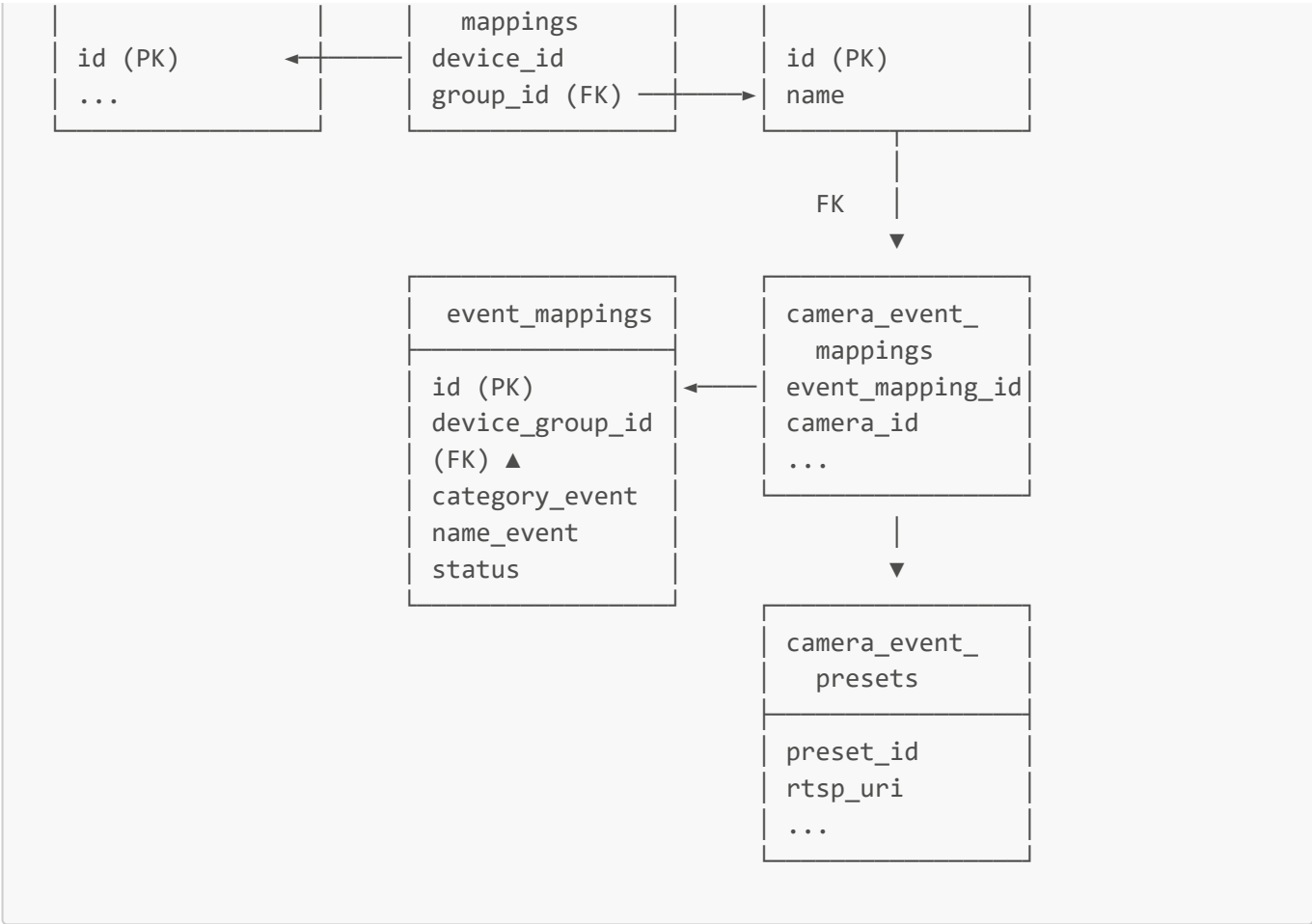


9.5 EventMapping ↔ DeviceGroup 관계

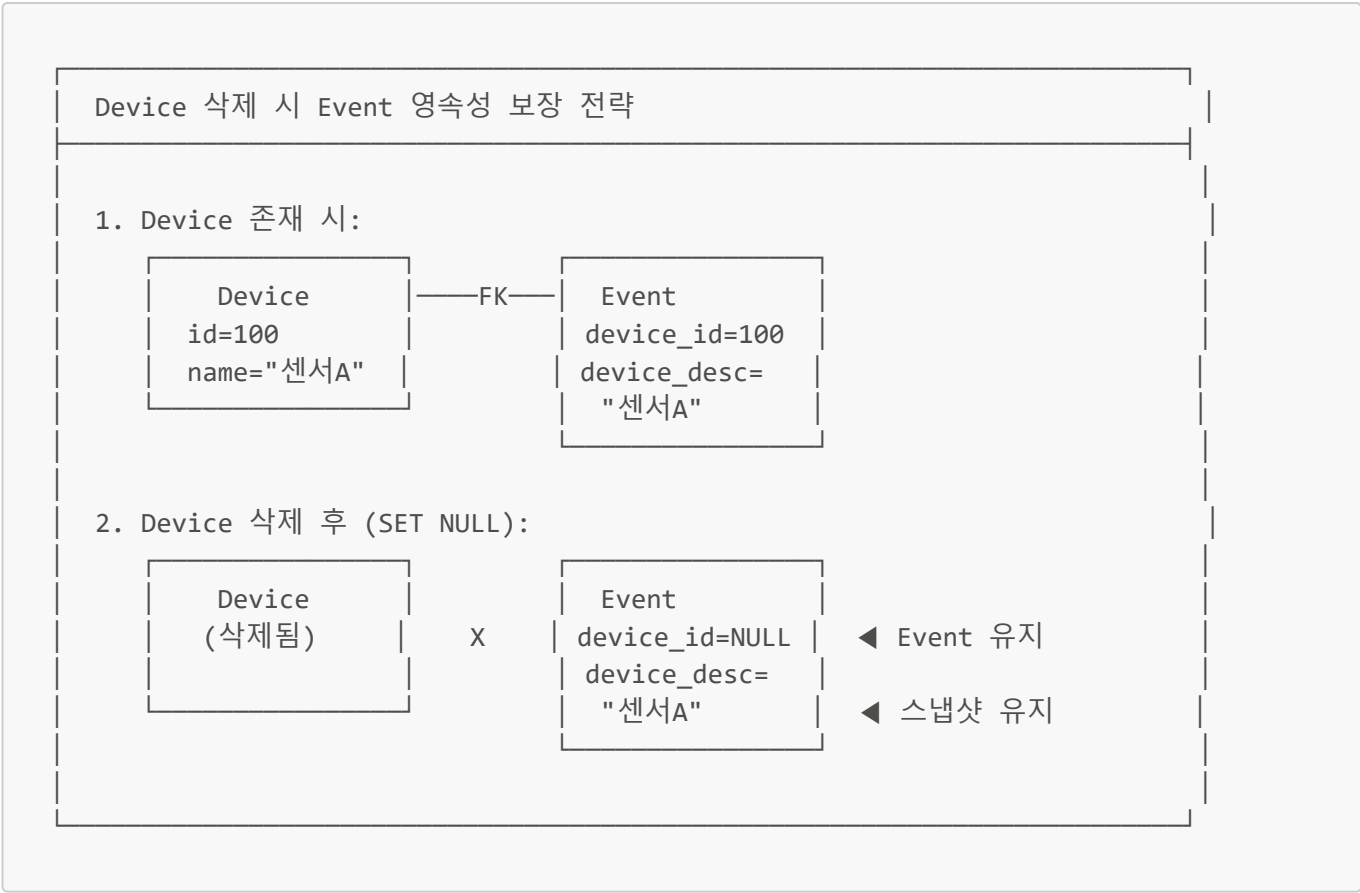
v1.2 변경사항: group_event (VARCHAR) → device_group_id (FK)

이벤트 발생 시 카메라 프리셋 연동 흐름:





9.6 Event 연속성 설계



10. 변경 이력

버전	날짜	변경 내용
v1.3	2026-01-06	API v2.3 반영 • events.sequence NULL 허용: 외부 시스템에서 제공되지 않을 수 있어 선택적으로 변경 • API 버전 참조 업데이트: v2.2 → v2.3
v1.2	2026-01-05	Event Joined Table Inheritance 적용 (PRD v2.1) • events Base Table 추가: Polymorphic Parent 테이블 • group_event 필드 제거: Event 테이블에서 불필요한 중복 필드 제거 • event_mappings 변경: group_event (VARCHAR) → device_group_id (FK) • action_events 변경: from_event + from_type_event → from_event_id (단일 FK) • Event Child 테이블 구조 변경: id가 FK/PK로 events.id 참조 • ERD 다이어그램 전면 수정: Joined Table Inheritance 구조 반영 • EventMapping ↔ DeviceGroup 관계 다이어그램 추가
v1.1	2025-12-31	Event 테이블 추가 • Event 관련 테이블 (detection_events, malfunction_events, connection_events, action_events) • PRD_Event_Device_Refactoring.md v1.1 적용 • Event 영속성 보장 (device_id SET NULL on delete) • device_description 스냅샷 필드 추가 • enum_detection_type, enum_fault_type Enum 추가 • Event ERD 다이어그램 추가
v1.0	2025-12-31	초기 스키마 문서 • Device 관련 테이블 (devices, controllers, sensors, cameras) • DeviceGroup 관련 테이블 (device_groups, device_group_mappings) • Camera Preset 관련 테이블 (camera_presets, rois, xy_points) • Integration 관련 테이블 (event_mappings, camera_event_mappings, camera_event_presets) • Server 관련 테이블 (server_categories, servers) • 전체 Enum 타입 정의 • ERD 다이어그램 추가

문서 종료